Dockerfile

构建镜像

`docker build -t <image_name> <image_file_path>`

From

指定基于的基础镜像

Run

执行指令，每次dockerfile指令都会在docker上新建一层，尽量减少dockerfile指令避免image过大

```
From ubuntu
RUN apt update \
    && git clone *** \
    && tar -xvfz ***.tar.gz
```

COPY

复制上下文目录或目录中文件到容器指定路径

`COPY --chown=<user>:<group> <源路径> <目标路径>` 可选参数，改变复制到容器内文件的拥有这和属组

```
COPY home /mydir/
COPY test.txt /mydir/
```

CMD

类似RUN指令，但RUN在docker build时运行

CMD在docker run时候运行

```
CMD ["<可执行文件或命令>","<参数1>","<参数2>"]
```

ENTRYPOINT

类似CMD，但不会被docker run时命令行参数指定覆盖，而会作为参数传入ENTRYPOINT

存在多个ENTRYPOINT，仅最后一个生效

ENTRYPOINT

类似于 CMD 指令，但其不会被 docker run 的命令行参数指定的指令所覆盖，而且这些命令行参数会被当作参数送给 ENTRYPOINT 指令指定的程序。

但是, 如果运行 docker run 时使用了 --entrypoint 选项，将覆盖 ENTRYPOINT 指令指定的程序。

优点：在执行 docker run 的时候可以指定 ENTRYPOINT 运行所需的参数。

注意：如果 Dockerfile 中如果存在多个 ENTRYPOINT 指令，仅最后一个生效。

格式：

```
ENTRYPOINT ["<executeable>","<param1>","<param2>",...]
```

可以搭配 CMD 命令使用：一般是变参才会使用 CMD ，这里的 CMD 等于是在给 ENTRYPOINT 传参，以下示例会提到。

示例：

假设已通过 Dockerfile 构建了 nginx:test 镜像：

```
FROM nginx

ENTRYPOINT ["nginx", "-c"] # 定参
CMD ["/etc/nginx/nginx.conf"] # 变参
```

1、不传参运行

```
$ docker run  nginx:test
```

容器内会默认运行以下命令，启动主进程。

```
nginx -c /etc/nginx/nginx.conf
```

2、传参运行

```
$ docker run  nginx:test -c /etc/nginx/new.conf
```

容器内会默认运行以下命令，启动主进程(/etc/nginx/new.conf:假设容器内已有此文件)

```
nginx -c /etc/nginx/new.conf
```

ENV

设置环境变量

```
ENV <key> <value>
ENV <key1>=<value1> <key2>=<value2>...
```

ARG

构建参数，与ENV作用一致，但作用域仅限于dockerfile（即只有docker build时可用）

EXPOSE

声明端口，帮助镜像使用者理解此镜像守护端口

```
EXPOSE <端口1> [<端口2>...]
```

`docker run -P` 随机端口映射时，会自动随即映射到EXPOSE端口

WORKDIR

指定工作目录。用 WORKDIR 指定的工作目录，会在构建镜像的每一层中都存在。（WORKDIR 指定的工作目录，必须是提前创建好的）。

docker build 构建镜像过程中的，每一个 RUN 命令都是新建的一层。只有通过 WORKDIR 创建的目录才会一直存在。

```
WORKDIR <工作目录路径>
```

USER

用于指定执行后续命令的用户和用户组，这边只是切换后续命令执行的用户（用户和用户组必须提前已经存在）。

```
USER <用户名>[:<用户组>]
```

ONBUILD

用于延迟构建命令的执行。简单的说，就是 Dockerfile 里用 ONBUILD 指定的命令，在本次构建镜像的过程中不会执行（假设镜像为 test-build）。当有新的 Dockerfile 使用了之前构建的镜像 FROM test-build ，这时执行新镜像的 Dockerfile 构建时候，会执行 test-build 的 Dockerfile 里的 ONBUILD 指定的命令。

```
ONBUILD <其它指令>
```

LABEL

LABEL 指令用来给镜像添加一些元数据（metadata），以键值对的形式，语法格式如下：

```
LABEL <key>=<value> <key>=<value> <key>=<value> ...
#添加镜像作者
LABEL org.opencontainers.image.authors="runoob"
```